

Introduction to LangChain



LangChain Ecosystem Overview

Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

The LangChain Ecosystem – Five Pillars

The modern LangChain ecosystem is a carefully structured suite of five interconnected pillars

1

**LangChain
Core**

Base abstractions,
interfaces & LCEL

2

LangChain

Chains, memory
& cognitive patterns

3

**Community &
Partners**

700+ integrations
with providers

4

LangGraph

Stateful agents
with graph control

5

LangSmith

Debugging, testing
& monitoring

What is LangChain?



What is LangChain?

Definition: An open-source framework for building LLM-powered applications with modular, composable building blocks.

What LangChain Provides

- ✓ Manages prompts & chains multiple reasoning steps
- ✓ Connects LLMs to external data sources
- ✓ Maintains conversation history & memory
- ✓ Orchestrates tools & handles errors gracefully
- ✓ Composable blocks that snap together like LEGO

The Origin Story

Oct 2022 Created by Harrison Chase

Feb 2023 \$10M seed round; JS version

Mar 2023 LangSmith announced

Jan 2024 LangGraph launched

2024-25 100K+ stars, 700+ integrations

LangChain is the lingua franca of LLM application development — its concepts define how the industry builds AI apps.

The Problem LangChain Solves



Building a Document Q&A System

✗ Without LangChain

1. Load PDFs & extract text (handle formats, OCR)

2. Implement chunking logic (sizes, overlaps)

3. Generate embeddings (batching, rate limits)

4. Set up vector DB, index & query logic

5. Build retrieval function for similarity search

6. Construct prompt with context + question

7. Call LLM API (streaming, token limits)

8. Parse response into structured format

9. Add conversation memory

10. Add logging and tracing

Result: 1000+ lines of tightly coupled, hard-to-maintain code

✓ With LangChain

```
from langchain_openai import ChatOpenAI
from langchain_chroma import Chroma
from langchain.chains import RetrievalQA

# Load, chunk, embed documents
vectorstore = Chroma.from_documents(docs)

# Build Q&A chain in one line
qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vectorstore.as_retriever()
)

result = qa.invoke("What is...?")
```

✓ ~30 lines of composable, modular code

✓ Each component independently testable & swappable

✓ Streaming, async & observability built in

The Five Pillars of LangChain

1

Pillar 1: LangChain Core

Foundation: Base abstractions, interfaces, and the LangChain Expression Language (LCEL). Zero heavy dependencies.

What It Contains

Base Interfaces: Language models, embeddings, vector stores, retrievers, loaders, parsers

Runnable Protocol: Universal interface: `invoke()`, `stream()`, `batch()`, `async` variants

LCEL: Declarative composition using the pipe (`|`) operator

Callback System: Hooks for logging, tracing, streaming, custom events

Message Types: `HumanMessage`, `AIMessage`, `SystemMessage`, `ToolMessage`

LCEL – Pipe Composition

```
# LCEL pipe syntax

chain = prompt | model | parser

# Every Runnable supports:
chain.invoke(input)      # sync
chain.stream(input)      # streaming
chain.batch([inputs])    # parallel
chain.ainvoke(input)     # async
```

Design Principle: Zero heavy dependencies – deliberately lightweight.

2

Pillar 2: LangChain (Chains & Memory)

Cognitive Architecture: Higher-level chains, agents, and retrieval strategies that encode proven thinking patterns.

Pre-built Chains

LLMChain — Prompt + model + output parser

RetrievalQA — Q&A over retrieved documents

ConversationalRetrievalChain — Chat-style doc Q&A with memory

MapReduceDocumentsChain — Map over chunks, reduce results

SummarizationChain — Progressive summarization

Memory Modules

BufferMemory — Stores full conversation history

SummaryMemory — LLM-generated conversation summaries

WindowMemory — Last K exchanges only

TokenBufferMemory — Token-limited history

EntityMemory — Tracks entities across conversation

Cognitive architectures — proven thinking patterns so you don't reinvent them.

3

Pillar 3: Community & Partner Packages

Swappability: 700+ integrations. Switching providers requires changing a single import – chain logic remains untouched.

Package	Provider	What it Provides
langchain-openai	OpenAI	ChatOpenAI, Embeddings, DALL-E
langchain-anthropic	Anthropic	ChatAnthropic (Claude models)
langchain-google-genai	Google	ChatGoogleGenerativeAI (Gemini)
langchain-groq	Groq	ChatGroq (fast inference)
langchain-ollama	Ollama	ChatOllama (local models)
langchain-pinecone	Pinecone	PineconeVectorStore
langchain-chroma	Chroma	Chroma vector store

Integration Coverage

50+ Chat Models	30+ Embeddings	50+ Vector Stores	160+ Doc Loaders	50+ Tools	20+ Retrievers
--------------------	-------------------	----------------------	---------------------	--------------	-------------------

4 Pillar 4: LangGraph

Why it exists: Chains are linear. Real agents need cycles — reason, act, observe, decide. This requires graphs, not chains.

What LangGraph Provides

State Machines: Persistent application state across steps

Nodes & Edges: Processing steps with conditional connections

Conditional Routing: Graph decides path based on current state

Persistence: Save and resume execution (checkpointing)

Human-in-the-Loop: Pause for human approval before continuing

Streaming: Stream intermediate results during execution

Subgraphs: Compose complex systems from reusable graphs

LangGraph vs Legacy Agents

Aspect	Legacy	LangGraph
Control Flow	Implicit	Explicit graph
State	Limited	Full persistence
Cycles	Basic retry	Complex loops
Human Oversight	Minimal	Built-in HITL
Debugging	Opaque	Visual graph
Production	Prototype	Production-grade

Controllability — explicit control over every decision point while leveraging LLM intelligence at each node.

5 Pillar 5: LangSmith

The Problem: When a chatbot gives a wrong answer — was it the prompt? The context? The model? LangSmith gives end-to-end visibility.

1

Tracing

Visualize every step — prompts, tokens, latency, outputs

2

Debugging

Click into any trace to see raw inputs/outputs per component

3

Evaluation

Automated evals — faithfulness, relevance, correctness at scale

4

Testing

Regression test suites — catch when prompt changes degrade quality

5

Monitoring

Production metrics — latency, cost, error rates, user feedback

6

Playground

Test prompts interactively with different models & parameters

Observability as a first-class concern — set an environment variable and every execution is traced automatically.

Package Installation & Alternatives



Package Installation Strategy

Key Principle: Install only what you need. The modular structure means you don't pull in 700+ integrations when you only need OpenAI and Chroma.

1

Basic LLM Apps

```
pip install langchain
langchain-core
langchain-openai
```

2

RAG Applications

```
pip install langchain
langchain-openai
langchain-chroma
langchain-community
```

3

Agent Applications

```
pip install langgraph
langchain-openai
```

4

Observability

```
pip install langsmith
```

LangChain vs The Alternatives

Framework	Primary Strength	Best For	Limitation
LangChain	Broadest ecosystem, most integrations	General-purpose LLM apps, RAG, chains	Abstraction overhead for simple tasks
LlamaIndex	Data ingestion and indexing	RAG-heavy applications	Less flexible for non-retrieval
Haystack	Production NLP pipelines	Enterprise search and QA	Smaller community
Semantic Kernel	Microsoft ecosystem	.NET/C# apps, Azure-centric	Limited outside Microsoft stack
Direct API Calls	Zero overhead	Simple, single-model tasks	No composition, memory, or tooling

Rule of thumb: Direct API calls for simple tasks. LangChain when you need chaining, retrieval, memory, or multi-provider support.

Key Takeaways

Key Takeaways

1

Five-Pillar Ecosystem — Core, LangChain, Community/Partners, LangGraph, and LangSmith — each solving a distinct part of the lifecycle

2

700+ Integrations — Partner & community packages enable connection to virtually any LLM provider, vector database, or data source

3

Modular Installation — Install only what you need — no bloated dependencies. Each package is independently versioned

4

Composability via LCEL — The Runnable protocol and pipe operator let you snap components together with streaming & async built in

5

Prototype to Production — LangGraph for agents and LangSmith for monitoring extend the ecosystem from prototyping into production

Understanding the LangChain ecosystem is essential — its concepts define how the entire industry builds AI-powered software.

Core Components



Models, Prompts & Output Parsers

Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

The Three Core Components

Every LLM application is built from three foundational building blocks – the minimum viable chain

1

Models

The AI Brain

Unified interface over
all LLM providers.
Write once, swap freely.

2

Prompts

The Instructions

Reusable, parameterized
templates that transform
fragile strings into objects.

3

Output Parsers

The Formatter

Transform raw LLM text
into structured, typed
data for your application.

Models = Stove (heat source) | Prompts = Recipe (instructions) | Output Parsers = Plating (presentation)

Component 1: Models

1

Models – Two Primary Types

Abstraction Layer: Unified interface over all LLM providers – write application logic once, swap models by changing a single line.

Type 1: LLMs (Legacy Text Completion)

Interface: Text in → Text out

Input: A single string (the prompt)

Output: A single string (the completion)

No concept of roles (system, user, assistant)

No built-in conversation structure

Largely deprecated in favor of Chat Models

```
# Legacy LLM
llm = OpenAI()
result = llm.invoke("Complete this...")
```

Type 2: Chat Models (Modern Standard)

Interface: Messages in → Message out

Input: A list of Message objects with defined roles

Output: An AIMessage object

Built-in system instructions & conversation history

Native streaming & tool call support

The standard for all new LLM applications

```
# Modern Chat Model
chat = ChatOpenAI(model="gpt-4o")
result = chat.invoke(messages)
```

Most modern providers have moved entirely to chat-based APIs – Chat Models are the standard.



Message Types & Unified Interface

Message Types in LangChain

Type	Role	Purpose	Example
SystemMessage	System	Sets behavior & constraints	"You are a helpful coding assistant"
HumanMessage	User	User's input or question	"How do I read a CSV?"
AIMessage	Assistant	Model's response	"Use pandas: pd.read_csv()"
ToolMessage	Tool	Results from tool/function calls	{"temperature": 72, "condition": "sunny"}

Common Methods on All Chat Models

invoke() Standard request-response

batch() Process multiple inputs in parallel

bind_tools() Attach tool definitions (function calling)

stream() Token-by-token for real-time UIs

ainvoke() / **astream()** Async versions for high concurrency

with_structured_output() Force output into a schema

Every Chat Model implements the same interface — swapping providers requires zero application code changes.



Supported Providers & Configuration

Provider	Package	Class	Key Strengths
OpenAI	langchain-openai	ChatOpenAI	GPT-4o, function calling, vision
Anthropic	langchain-anthropic	ChatAnthropic	Claude 3.5/4, 200K context, safety
Google	langchain-google-genai	ChatGoogleGenerativeAI	Gemini, multimodal, free tier
Groq	langchain-groq	ChatGroq	Ultra-fast inference (500+ tok/s)
Ollama	langchain-ollama	ChatOllama	Local models, no API key, privacy

Temperature Guidance

0.0

Deterministic — data extraction, classification, code gen

0.3–0.5

Balanced — summarization, Q&A, general assistance

0.7–0.9

Creative — brainstorming, creative writing, suggestions

1.0+

Maximum creativity — poetry, experimental generation

Provider switching: Change the import and initialization — prompts, chains, memory, parsers all remain identical.



Model-Specific Features

Unified + Unique: While the interface is unified, each provider offers unique capabilities that LangChain exposes.

1

OpenAI

- ✓ Vision input (image analysis with GPT-4o)
- ✓ JSON mode (guaranteed JSON output)
- ✓ Seed parameter (reproducible outputs)
- ✓ Parallel function calling

2

Anthropic

- ✓ Extended thinking (chain-of-thought)
- ✓ 200K token context window
- ✓ Document & image understanding
- ✓ System prompt caching

3

Google

- ✓ Multimodal (text, images, audio, video)
- ✓ Grounding with Google Search
- ✓ Code execution capabilities

4

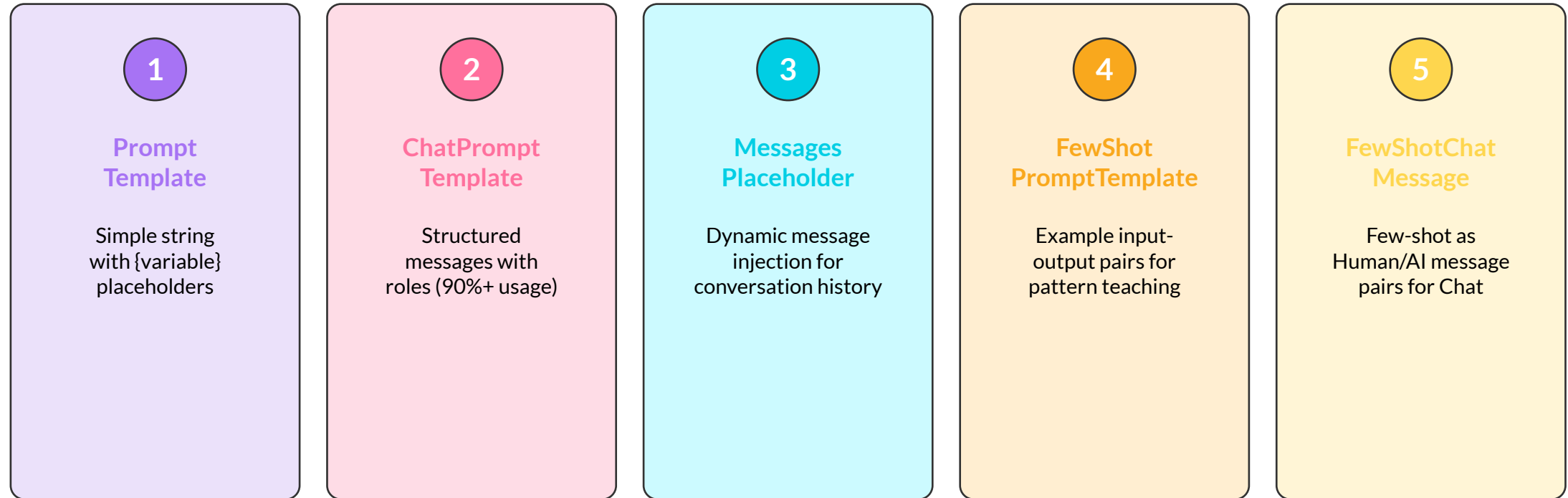
Ollama

- ✓ Fully local execution (no internet)
- ✓ Custom model loading (GGUF files)
- ✓ No API costs
- ✓ Complete data privacy

Component 2: Prompts

2 Prompts – The Instructions

Why Templates? Raw strings break down in real apps – hard to reuse, test, maintain, compose. Templates treat prompts as first-class objects.



ChatPromptTemplate with role-based messages is the standard for modern applications.



Prompt vs ChatPrompt Template

PromptTemplate (Simple String)

```
from langchain_core.prompts import \
    PromptTemplate

template = PromptTemplate(
    template="Translate {text} to {lang}"
    input_variables=["text", "lang"]
)

prompt = template.invoke({"text": "Hello",
    "lang": "Tamil"})
```

- ✓ Variable substitution with {name} syntax
- ✓ Input validation — errors if vars missing
- ✓ Partial variables — pre-fill some, fill rest later

ChatPromptTemplate (Modern Standard)

```
from langchain_core.prompts import \
    ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are {role}"),
    ("human", "{question}"),
])

msgs = prompt.invoke({"role": "tutor",
    "question": "Explain RAG"})
```

- ✓ Role-based: System, Human, AI messages
- ✓ Message tuple shorthand for rapid definition
- ✓ Composability — combine multiple templates

Tuple shorthand: ("system", "...") → SystemMessage | ("human", "...") → HumanMessage | ("placeholder", "{var}") → Message list



Placeholder & FewShot Templates

MessagesPlaceholder

Injects a variable-length list of messages at runtime — essential for conversation history.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a tutor"),
    # Inject conversation history
    ("placeholder", "{history}"),
    ("human", "{question}"),
])
```

✓ Conversation history injection

✓ Agent scratchpads (reasoning steps)

✓ Few-shot examples as messages

✓ Dynamic context injection

FewShotPromptTemplate

Includes example input-output pairs to teach the model the desired behavior pattern.

Components

Example Selector: Chooses which examples to include (static or dynamic)

Example Prompt: Template for formatting each individual example

Prefix: Text before examples (task description)

Suffix: Text after examples (actual input)

Best For

- ✓ Tasks where zero-shot performance is poor
- ✓ Complex extraction or classification tasks

Dynamic example selection: `SemanticSimilarityExampleSelector` shows the most relevant examples for each query.



Prompt Template Best Practices

1

Always use templates over raw strings — Even for simple prompts — enables testing, reuse, and versioning

2

Name variables descriptively — {user_question} is clearer than {q} or {input}

3

Keep system prompts separate — Define system prompts as their own template for easy swapping

4

Use partial variables for static context — Pre-fill date, model name, app version at startup

5

Version your templates — Store templates in files or version control, not inline code

6

Test templates independently — Templates are pure functions — test them without calling the LLM

7

Use few-shot for complex tasks — When the model struggles with zero-shot, add 2–3 examples

Component 3:

Output Parsers

3

Output Parsers – The Response Formatter

Core Problem: LLMs generate text. Your application needs data. Output Parsers bridge the gap between unstructured text and structured, typed data.

What LLMs Actually Return

Perfect JSON: {"name": "John", "age": 30}

JSON wrapped in markdown code fences

"Here's the output:" + preamble text + JSON

Slightly malformed (trailing commas, missing quotes)

Natural language response instead of JSON

The Parser Interface

`get_format_instructions()`

Returns a string describing the expected format — inject into your prompt so the model knows what to produce

`parse()`

Takes raw string output from LLM and converts it into the target data structure

Output Parsers provide: Format instructions (for the prompt) + Parsing logic (for the response) + Error recovery.

Output Parser Types – Comparison

Parser	Output Type	Validation	Best For	Complexity
StrOutputParser	string	None	Plain text responses	Minimal
JsonOutputParser	dict	Basic JSON	Quick structured data	Low
PydanticOutputParser	Pydantic model	Full type + schema	Production applications	Medium
CommaSeparatedList	list	Basic split	Simple lists	Minimal
StructuredOutputParser	dict	Field-based	Prototyping	Low
XMLOutputParser	dict	XML structure	XML-native workflows	Low
EnumOutputParser	enum value	Strict enum match	Classification tasks	Minimal

PydanticOutputParser = gold standard for production | StructuredOutputParser = quick prototyping | StrOutputParser = plain text



Structured Output (Modern)

model.with_structured_output(): Uses the model's native function calling / tool use to guarantee output matches your schema — no parsing errors.

Advantages over Traditional Parsers

✓ **More Reliable:** Uses model's built-in JSON mode or function calling

✓ **No Parsing Errors:** Model is constrained to produce valid output

✓ **Simpler Code:** No need to manage format instructions separately

✓ **Flexible Schema:** Works with Pydantic models, TypedDict, or JSON schema

```
# Modern approach
structured_llm = model.with_structured_output(
    MovieReview # Pydantic model
)

result = structured_llm.invoke("Review Inception")
```

When to Still Use Output Parsers

No Function Calling: Models that don't support native structured output

Custom Parsing Logic: Beyond schema validation (complex transformations)

Non-JSON Formats: XML, comma-separated lists, custom formats

Prompt Transparency: Need format instructions visible in the prompt

Decision Guide

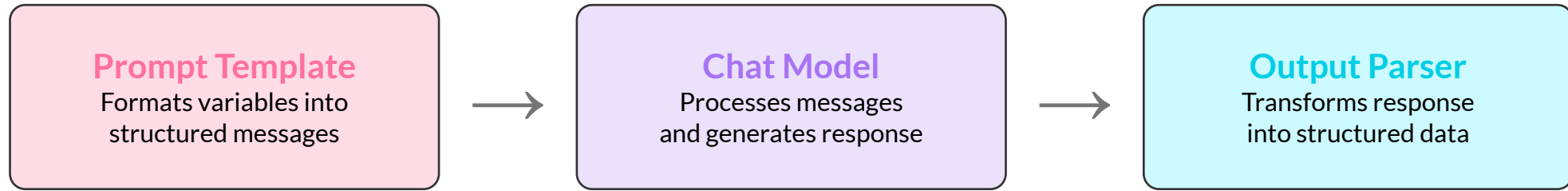
Model supports structured output?

→ **Yes:** Use with_structured_output()

→ **No / Custom format needed:** Use Output Parsers

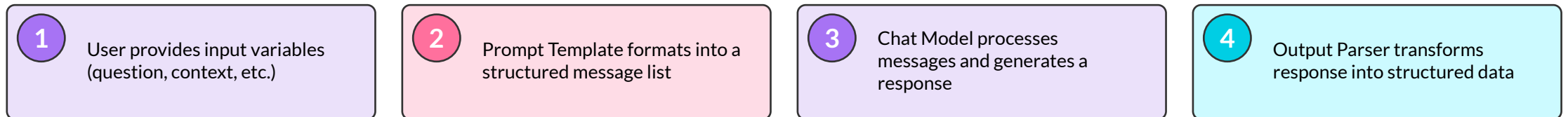
How They
Work Together

The Fundamental Chain Pattern



```
# LCEL Syntax – the pipe operator chains components
chain = prompt | model | parser
result = chain.invoke({"question": "Explain RAG"})
```

Data Flow



This three-component chain is the atom of LangChain — every complex application is built by combining these atoms.

Key Takeaways

Key Takeaways

1

Models – Unified Interface — Write once, swap freely. Chat Models (message-based) are the modern standard; legacy LLMs are deprecated.

2

Prompt Templates — Transform fragile strings into reusable, testable, composable objects. ChatPromptTemplate is the standard.

3

Output Parsers — Bridge unstructured text to structured data. PydanticOutputParser for production; with_structured_output() for native reliability.

4

The Chain Pattern — prompt → model → parser is the fundamental building block. Master this, and RAG, agents, complex chains become natural extensions.

5

Abstraction Without Lock-in — Clean, consistent API while retaining ability to switch any component — different model, prompt strategy, or output format.

Models (the brain) + Prompts (the instructions) + Output Parsers (the formatter) = The minimum viable chain.

LangChain Expression Language (LCEL)



The Declarative Composition Framework

Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover

A complete guide to LangChain's declarative composition framework for building LLM pipelines

1

LCEL & Runnable Protocol

What LCEL is, why it exists, and the universal Runnable interface

2

Pipe Operator & Primitives

Building chains with | and the five essential LCEL building blocks

3

Advanced Patterns

RAG chains, multi-chain routing, context accumulation, model comparison

4

Streaming & Batching

Automatic streaming, batch processing, and event-driven execution

5

Error Handling & Config

Retry logic, fallbacks, runtime configuration, and LangSmith tracing

What is LCEL?



What is LCEL?

LCEL: LangChain Expression Language — the declarative composition framework for building complex LLM pipelines by chaining components with the pipe (|) operator.

The Plumbing Analogy

- Individual components (prompts, models, parsers) are like faucets, pipes, and filters
- LCEL connects them into a functioning pipeline where data flows seamlessly
- Once connected, the entire pipeline inherits powerful capabilities automatically
- Streaming, async, batching, error handling, and observability — with zero additional code

Built-in Capabilities



Streaming

Token-by-token output

Async Support

Every chain is async-capable

Batch Processing

Automatic parallelization

Error Handling

Retry & fallback logic

Observability

Auto-traced in LangSmith



Why LCEL Exists

The Problem: Before LCEL, building LangChain pipelines required imperative code — manually calling each component, passing outputs, and handling errors at every step.

Imperative Approach (Pre-LCEL)

- ✗ Verbose boilerplate for every pipeline
- ✗ Streaming required custom implementation
- ✗ Async meant rewriting entire pipeline
- ✗ No automatic retry or fallback logic
- ✗ No built-in tracing — manual logging
- ✗ Batch processing needed custom code
- ✗ Changing structure = rewriting code



LCEL Approach (Declarative)

- ✓ Pipelines defined in 1–3 lines of code
- ✓ Streaming works automatically end-to-end
- ✓ Every chain is automatically async-capable
- ✓ Retry & fallback as chain modifiers
- ✓ Auto-traced in LangSmith — zero config
- ✓ Batch with automatic parallelization
- ✓ Swap or reorder components easily

The Runnable Protocol

1 The Runnable Interface

Runnable Protocol: Every LCEL component implements the Runnable interface — the universal contract that makes all components composable through the pipe operator.

Method	Signature	Description	Use Case
<code>invoke()</code>	<code>input → output</code>	Process a single input synchronously	Standard request-response
<code>stream()</code>	<code>input → Iterator[output]</code>	Yield results incrementally	Real-time chat UIs
<code>batch()</code>	<code>[inputs] → [outputs]</code>	Process multiple inputs in parallel	Bulk document processing
<code>ainvoke()</code>	<code>input → output (async)</code>	Async version of <code>invoke</code>	High-concurrency servers
<code>astream()</code>	<code>input → AsyncIterator</code>	Async streaming	Async chat applications
<code>abatch()</code>	<code>[inputs] → [outputs] (async)</code>	Async batch processing	Large-scale parallel tasks

When you compose Runnables with LCEL, the resulting chain is itself a Runnable — `invoke`, `stream`, or `batch` the entire chain with one call.

2

Common Runnable Types & I/O

Type Safety: Each Runnable has defined input/output types. The output type of one component must be compatible with the input type of the next in a chain.

Component	Input Type	Output Type
ChatPromptTemplate	dict (template variables)	ChatPromptValue (messages)
ChatModel	ChatPromptValue / list[Message]	AIMessage
StrOutputParser	AIMessage	string
JsonOutputParser	AIMessage	dict
PydanticOutputParser	AIMessage	Pydantic model instance
VectorStoreRetriever	string (query)	list[Document]
RunnableLambda	any	any (depends on function)
RunnablePassthrough	any	same as input (pass-through)

Pipe Operator & LCEL Primitives

I The Pipe Operator

Pipe Operator: The pipe (|) connects the output of one Runnable to the input of the next, creating a sequential data pipeline.

The Fundamental Chain Pattern

```
# Basic structure
chain = component_1 | component_2 | component_3

# The universal pattern
chain = prompt | model | output_parser
```

Data Flow

- 1 User provides dict variables
- 2 Prompt formats into messages
- 3 Model generates AIMessage
- 4 Parser extracts usable data
- 5 Final result returned

Extending the Chain

```
chain = prompt | model | parser | post_processor | formatter # Each | adds a processing step
```

The chain remains a single Runnable that can be invoked, streamed, or batched as a unit.

1-2 RunnablePassthrough & RunnableParallel

1 RunnablePassthrough

What it does: Passes input through unchanged. Essential for preserving data when you need it downstream.

Why it matters: In complex chains, you often need the original input at a later stage (e.g., the user's question after retrieval).

.assign() variant: Keeps original input AND adds new keys. Input `{"question": "..."} with assign(context=retriever) produces {"question": "...", "context": [docs]}.`

```
# RAG: preserve question + retrieve
{"context": retriever,

"question": RunnablePassthrough()}
```

2 RunnableParallel

What it does: Routes a single input to multiple Runnables that execute simultaneously. Collects outputs into a dictionary.

Why it matters: Many workflows require multiple independent operations on the same input at once.

Practical Applications:

- RAG: Retrieve docs + pass question simultaneously
- Multi-model comparison: Same prompt to GPT-4, Claude, Gemini
- Feature extraction: Different features in parallel
- Guard rails: Safety checks alongside main processing

```
# Dict shorthand = RunnableParallel
{"context": retriever,

"question": RunnablePassthrough()}
```


3-5 Lambda, Branch & Fallbacks

3 RunnableLambda

What: Wraps any Python function (sync or async) into a Runnable, making it composable in LCEL chains.

Why: Inject custom logic — data transformation, validation, API calls — anywhere in your chain.

How: @chain decorator: Syntactically cleaner alternative for custom chain steps.

Use Cases:

Data transformation • Formatting • Filtering • External APIs • Logging

4 RunnableBranch

What: Routes input to different Runnables based on conditions. Like an if/elif/else for chains.

Why: Real apps need different processing paths based on input type or content.

How: Evaluates conditions in order, routes to first True match, default fallback.

Use Cases:

Query routing • Language detection • Complexity routing • Error handling

5 RunnableWithFallbacks

What: Wraps a Runnable with fallback alternatives. If primary fails, automatically tries the next.

Why: LLM APIs are unreliable — rate limits, outages, timeouts. This provides automatic resilience.

How: Tries primary → catches exception → tries fallbacks in order → transparent to caller.

Use Cases:

Model cascade • Cache strategy • Graceful degradation • Format fallback

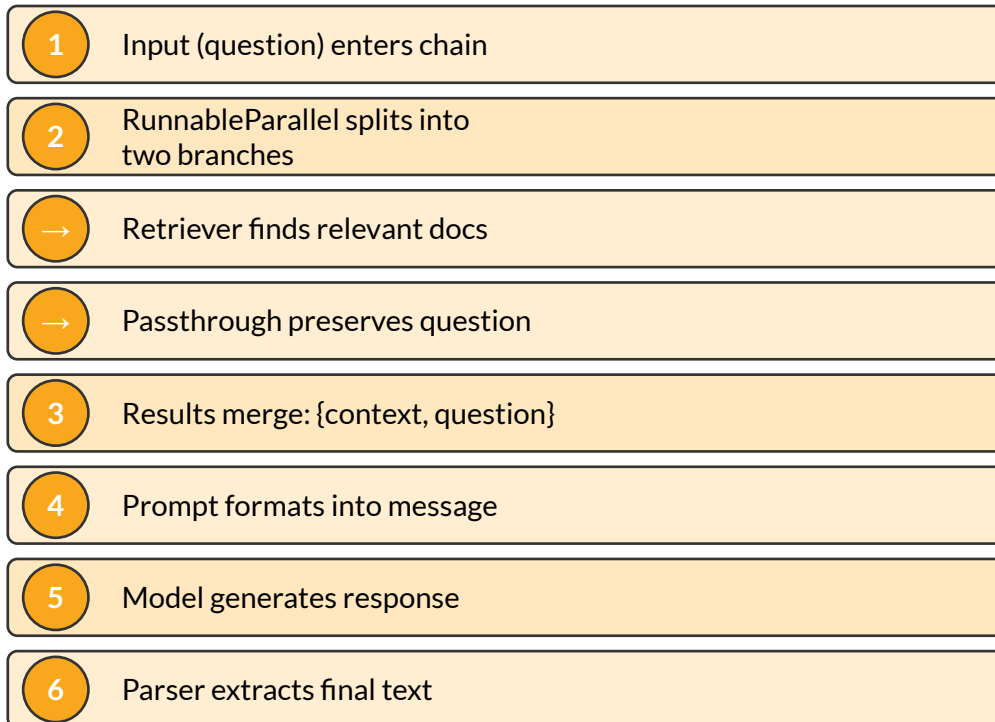
Advanced LCEL Patterns



Pattern 1: RAG Chain with LCEL

RAG Pattern: The most important LCEL composition — combining RunnableParallel, RunnablePassthrough, and sequential chaining for retrieval-augmented generation.

Architecture Flow



LCEL Implementation

```
# RAG Chain in LCEL
from langchain_core.runnables import (
    RunnablePassthrough,
    RunnableParallel
)

rag_chain = (
    {"context": retriever,
     "question": RunnablePassthrough() }

    | prompt

    | model

    | StrOutputParser()
)

# Single Runnable — invoke/stream/batch!
result = rag_chain.invoke("What is LCEL?")
```

This entire pipeline is a single Runnable — invoke it, stream it, or batch it with one method call.



More Advanced Patterns

2 Multi-Chain Routing

Route different query types to specialized chains:

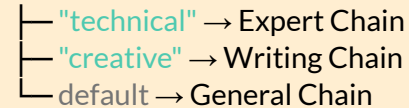
Input (query)



Classifier Chain → category



RunnableBranch



3 Context Accumulation

Build up context progressively through multi-step analysis:

Input (document)



Step 1: Summarize → summary



Step 2: Extract Key Points
(using original + summary)



Step 3: Generate Report
(using all previous data)

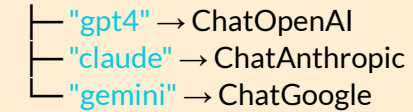
4 Parallel Model Comparison

Compare multiple models on the same prompt:

Input (question)



RunnableParallel



Aggregation → Best Response

Streaming, Batching & Error Handling



Streaming with LCEL

Automatic Streaming: When you call `.stream()` on an LCEL chain, every compatible component streams its output token-by-token — no additional code needed.

Without vs With LCEL Streaming

Without: Wait 5 seconds for entire response → complete answer appears at once

With: Each token passed through parser immediately → builds character by character

Streaming Events (`.astream_events()`)

on_chain_start / end: Chain-level lifecycle events

on_llm_start / stream / end: Model token-by-token events

on_retriever_start / end: Retrieval lifecycle events

on_tool_start / end: Tool execution events

Streaming Behavior by Component

Component	Streaming Behavior
Chat Model	Yields tokens as generated
StrOutputParser	Passes each token immediately
JsonOutputParser	Accumulates until valid JSON
PydanticOutputParser	Accumulates until complete
RunnablePassthrough	Passes through immediately
Retriever	Returns complete (no partial)

Components that can produce partial results stream naturally. Others accumulate and pass through once complete.



Batch Processing & Error Handling

Batch Processing

.batch() processes multiple inputs in parallel using Python threading for I/O-bound LLM calls.

Parameter	Default	Description
max_concurrency	None	Maximum parallel executions
return_exceptions	False	Return exceptions for partial success

- Processing multiple documents through the same chain
- Generating multiple content variations
- Evaluating chains across test datasets
- Bulk classification or extraction tasks

Error Handling

Retry Logic (.with_retry())

Automatic retries with exponential backoff and jitter for transient API errors.

```
model.with_retry(  
    stop_after_attempt=  
    3)
```

Fallback Chains (.with_fallbacks())

If primary fails, automatically tries fallback models in order. Transparent to caller.

```
model.with_fallbacks(  
    [fallback_1, fallback_2])
```

Combined: Retry + Fallback

Most resilient pattern – retries primary twice, then retries fallback twice

```
primary.with_retry(2)  
    .with_fallbacks([fallback.with_retry(2)])
```



Runtime Configuration & LangSmith

Configurable Fields

`.configurable_fields()` — Make parameters adjustable at runtime without rebuilding chains.

- Switch creative (high temp) vs factual (low temp) modes
- Adjust max_tokens based on the task
- Toggle streaming on/off at runtime

Configurable Alternatives

`.configurable_alternatives()` — Swap entire components at runtime (e.g., model providers).

- A/B testing different models in production
- User preference for specific models
- Cost-based routing (cheap → expensive)

LangSmith Integration

Automatic Tracing: Every LCEL chain is traced in LangSmith with zero additional code.

What Gets Traced

✓ **Component I/O:** Exact inputs and outputs for every step

✓ **Token Usage:** Usage and latency per component

✓ **Execution Time:** Total chain execution duration

✓ **Error Details:** Failure information when errors occur

✓ **Nested Traces:** Sub-chain execution visibility

Debugging LLM apps is hard. With automatic tracing, you can see exactly which component failed, what it received, and what it produced.

Best Practices & Comparisons



LCEL Best Practices

✓ **Keep chains simple:** Start with prompt → model → parser; add complexity only when needed

✓ **Name your chains:** Use `.with_config(run_name="name")` for easier debugging in LangSmith

✓ **Use `assign()` for context:** `RunnablePassthrough.assign()` is cleaner than manual dictionary construction

✓ **Prefer dictionary shorthand:** `{"key": runnable}` is cleaner than explicit `RunnableParallel`

✓ **Always add fallbacks:** LLM APIs fail — plan for it with `.with_fallbacks()`

✓ **Use retry with backoff:** Handles transient errors gracefully with `.with_retry()`

✓ **Set `max_concurrency`:** Respect API rate limits when using `.batch()`

✓ **Stream for user-facing apps:** Users perceive streaming as faster even when total time is the same

✓ **Test with `invoke()` first:** Easier to debug synchronous execution before adding streaming

✓ **Use `configurable_alternatives()`:** Don't hardcode model choices — enable runtime swapping

LCEL vs Imperative Chains

Aspect	LCEL (Declarative)	Imperative Code
Readability	Excellent for linear/parallel flows	Better for complex branching
Streaming	Automatic, first-class support	Manual implementation required
Async	Built-in for every chain	Must rewrite with async/await
Batching	One method call with parallelization	Custom threading/multiprocessing
Tracing	Automatic LangSmith integration	Manual logging
Error Handling	Declarative retry/fallback	Try/except blocks
Best For	Standard LLM pipelines, RAG, most apps	Dynamic logic, complex state mgmt
Learning Curve	Medium (Runnable protocol)	Low (standard Python)

Use LCEL for 90% of LLM chains. Fall back to imperative only for highly dynamic workflows — or consider LangGraph.

Key Takeaways

Key Takeaways

- 1 **LCEL is the composition framework:** Chain components with the pipe (|) operator. Every component implements the Runnable protocol.
- 2 **Five LCEL primitives:** Passthrough, Parallel, Lambda, Branch, and Fallbacks handle all common composition patterns.
- 3 **Streaming & batching are automatic:** Call .stream() or .batch() on any chain — every compatible component works seamlessly.
- 4 **Error resilience is declarative:** .with_retry() adds backoff, .with_fallbacks() provides automatic failover to alternative models.
- 5 **Master the RAG chain pattern:** RunnableParallel for retrieval + passthrough → prompt → model → parser. Foundation of production apps.

LCEL transforms complex LLM pipeline construction from imperative boilerplate into elegant, declarative composition.

Chains



Connecting Components for Complex Workflows

Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

Chain Categories – A Mental Model

All chains fall into one of these five categories – the building blocks of every LLM workflow

1

Sequential

Linear pipeline.
 $A \rightarrow B \rightarrow C \rightarrow \text{Output}$.
Output of each step
feeds the next.

2

Parallel

Concurrent branches.
Same input processed
simultaneously,
results merged.

3

Conditional

Route by condition.
Input directed to
different paths based
on runtime logic.

4

Map-Reduce

Process many items
(map), then combine
results (reduce).
Scales to any size.

5

Recursive

Loop until done.
Chain calls itself
or repeats until
condition is met.

Chains are the bridge between individual components and real-world applications – they turn components into functional software.

Evolution of Chains

1 Legacy Chains vs LCEL (Modern)

Generation 1: Legacy Chain Classes

LLMChain — **Deprecated**

SequentialChain — **Deprecated**

TransformChain — **Deprecated**

RouterChain — **Deprecated**

ConversationalRetrievalChain — **Migrating**

RetrievalQA — **Migrating**

Problems with Legacy Chains

- ✗ Rigid structure — hard to customize
- ✗ Opaque internals — difficult to debug
- ✗ No streaming — blocks until completion
- ✗ No async — synchronous only
- ✗ Inconsistent interfaces

Generation 2: LCEL (Current Standard)

```
# LCEL: Declarative & Composable
chain = prompt | model | parser

# Invoke, stream, batch — all built in
result = chain.invoke({...})
```

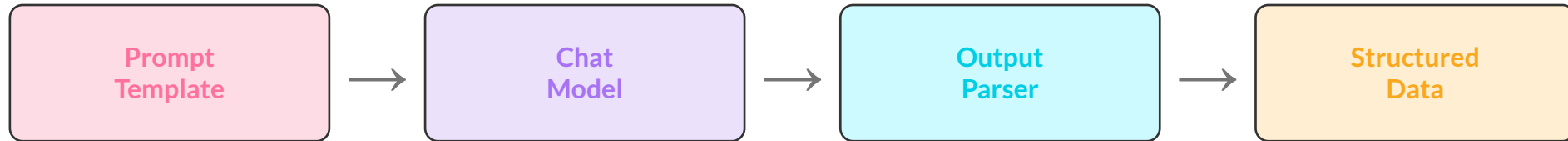
- ✓ Declarative & composable (pipe operator)
- ✓ Automatically streamable
- ✓ Async-capable & batchable
- ✓ Fully observable (LangSmith tracing)
- ✓ Customizable at every level
- ✓ Consistent interface (.invoke / .stream / .batch)

Use LCEL for all new development. Legacy chains are mentioned only for context when encountered in existing code.

Sequential & Parallel Chains

1 Sequential Chains

Linear Pipeline: Components execute one after another. Output of each step feeds directly into the next.



Context Accumulation with assign()

Start: `{"topic": "AI in healthcare"}`

After Step 1: `+ "search_queries": [...]`

After Step 2: `+ "documents": [...]`

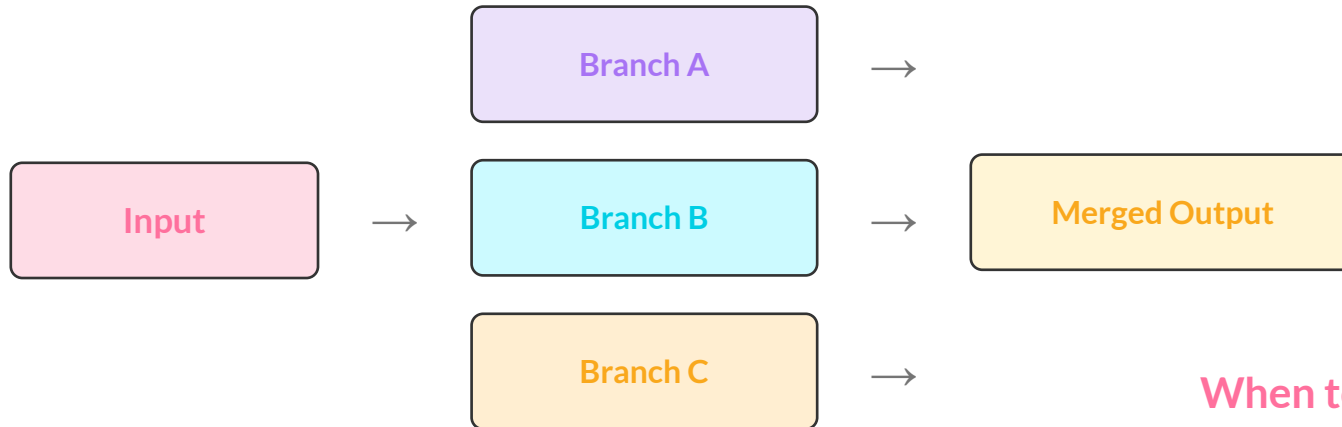
After Step 3: `+ "summary": "..."`

Key insight: Each step has access to ALL previous outputs via `assign()`, not just the last one.

2

Parallel Chains

Concurrent Execution: Multiple operations run simultaneously on the same input. Results are collected and merged.



Performance Benefits

Sequential: ~6 seconds (2s + 2s + 2s)

Parallel: ~2 seconds

3x faster

When to Use

- ✓ RAG: Retrieve docs AND pass through question
- ✓ Multi-aspect analysis (sentiment + entities + topic)
- ✓ Redundancy: Same query to multiple models
- ✓ Enrichment: Fetch data from multiple sources

Parallelization can cut response time by 50–80% for pipelines with retrieval + multiple LLM calls.

Conditional Chains

3

Conditional Chain Patterns

Routing Logic: Input is directed to different processing paths based on runtime conditions.

1

RunnableBranch

Explicit Routing

Evaluates conditions in order:

condition_1 → chain_1
condition_2 → chain_2
default → default_chain

Example: Language-aware support

Tamil → Tamil chain
Technical → Expert chain
Default → General chain

2

LLM-Based Routing

Dynamic Classification

LLM classifies input, then routes:

Input → Classifier → category
→ Branch by category

Advantages:

- ✓ Handles ambiguous inputs
- ✓ No complex regex rules
- ✓ Adapts to new categories
- ✓ Multilingual support

3

Semantic Router

Embedding Similarity

Uses embedding similarity:

1. Define route embeddings
2. Embed incoming query
3. Find most similar route
4. Route to matching chain

Advantages over LLM routing:

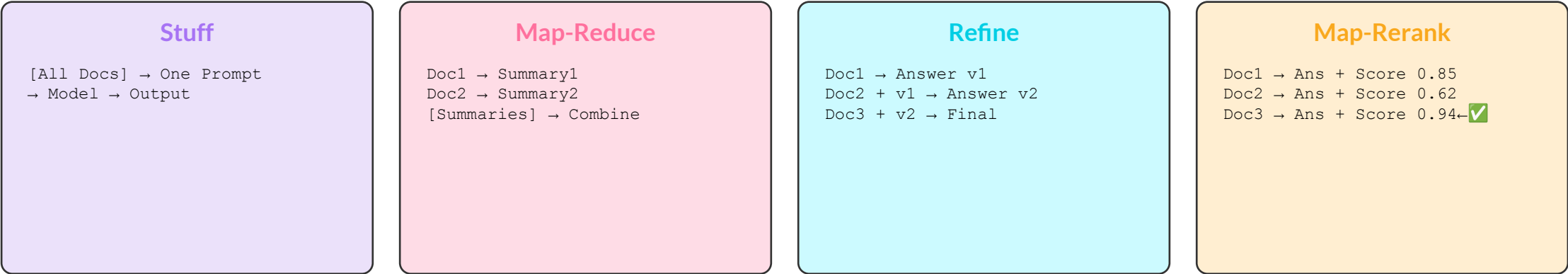
- ✓ Much faster (no LLM call)
- ✓ Lower cost & deterministic

Document Processing Chains

Document Processing Strategies

Strategy	How It Works	Speed	Quality	Cross-Doc	Best For
Stuff	All docs into one prompt	Fast	High (if fits)	Yes	Small doc sets
Map-Reduce	Process each doc, then combine	Fast (parallel)	Medium	Limited	Large collections
Refine	Sequential refinement per doc	Slow	Highest	Excellent	Detailed analysis
Map-Rerank	Process each, score, pick best	Fast (parallel)	Medium	None	Factual Q&A

Architecture Patterns



Conversational Chains



Conversational Chains & Memory

The Challenge: LLMs are stateless – each API call is independent. Memory must be explicitly managed and injected.

Memory Type	How It Works	Context Growth	Best For
Buffer Memory	Stores all messages verbatim	Grows unbounded	Short (<20 turns)
Window Memory	Keeps last K message pairs	Fixed ($K \times 2$)	Medium conversations
Token Buffer	Keeps messages up to N tokens	Fixed (N tokens)	Token-budget-aware
Summary Memory	LLM summarizes older messages	Grows slowly	Long conversations
Summary Buffer	Summary of old + verbatim recent	Hybrid growth	Best balance overall
Entity Memory	Tracks entities across conversation	Per-entity storage	Entity-heavy chats

Quick Selection Guide

1–10 turns
Buffer Memory

10–50 turns
Window / Token Buffer

50+ turns
Summary Buffer Memory

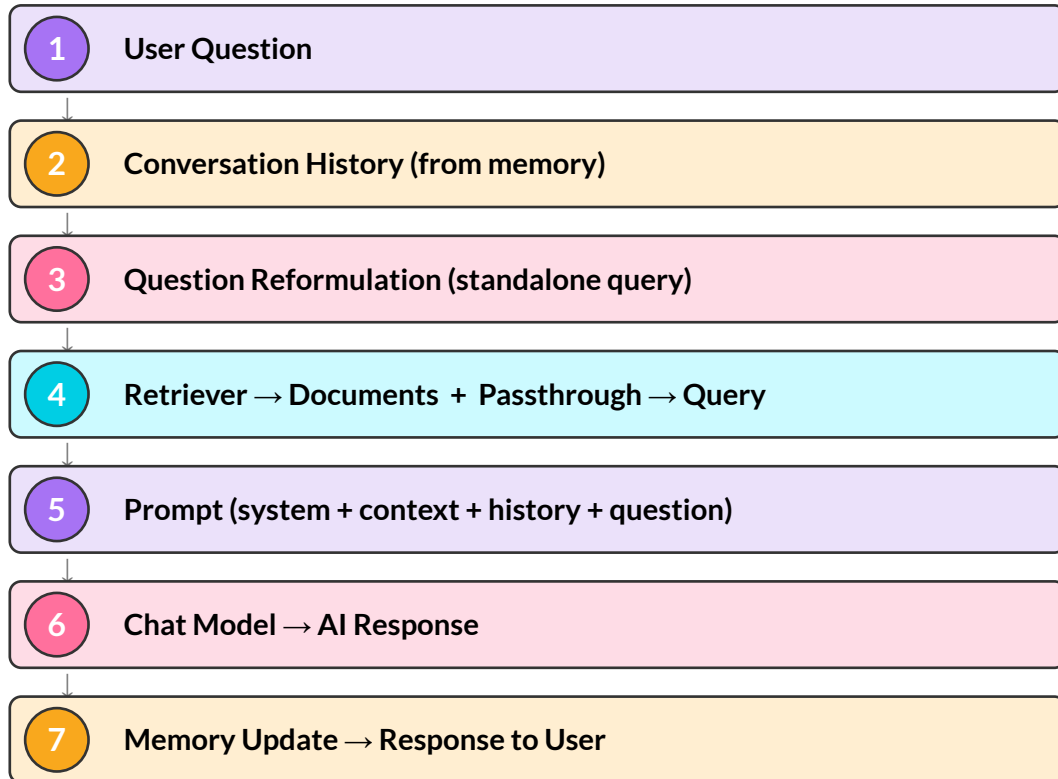
Entity-heavy
Entity Memory

Summary Buffer Memory provides the best balance – summarize old context, keep recent verbatim.



Conversational RAG Chain

Most Powerful Pattern: Combines memory with retrieval – maintains conversation state while accessing external knowledge.



Why Reformulation Matters

```
# Turn 1:
"What is LangChain?"
→ Clear query, works directly

# Turn 2:
"Who created it?"
→ "it" = LangChain (ambiguous!)

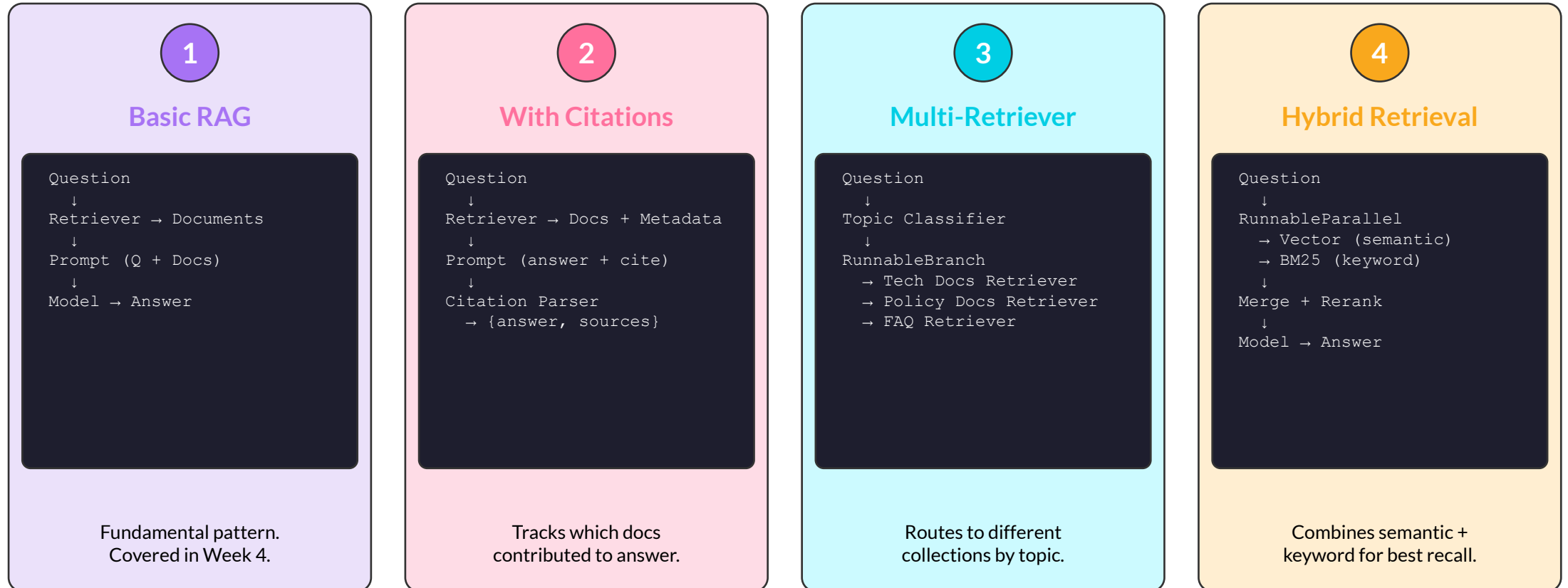
# Reformulated:
"Who created LangChain?" ✓
```

Without reformulation:

Follow-up questions with pronouns or implicit references will fail – retriever won't find relevant docs for queries like "Who created it?"

Retrieval Chains (RAG Patterns)

Retrieval Chain Patterns



Hybrid retrieval: Semantic search catches meaning, keyword search catches exact matches — combining both catches everything.

Chain Composition & Best Practices



Chain Composition Patterns

1

Nested Composition

Chains of Chains

Build complex systems from simple, tested building blocks.

```
research_chain = prompt|model|parser
outline_chain = prompt|model|parser
writing_chain = prompt|model|parser

full_pipeline =
  assign(research=research_chain)
  | assign(outline=outline_chain)
  | writing_chain
```

Each sub-chain is independently testable and reusable.

2

Human-in-the-Loop

Approval Gates

Insert human review at critical decision points:

```
Input → Draft Chain → Draft
↓
Human Review:
✓ Approve → Finalize
⇒ Edit → Re-process
✗ Reject → End
```

Better served by LangGraph for true interrupt/resume.

3

Self-Correcting

Quality Validation Loop

Chain checks its own output and retries if insufficient:

```
Input → Generate → Output
↓
Validation Chain → Pass/Fail
↓
If Pass → Return
If Fail → Regenerate
(with feedback)
```

True iterative correction is better handled by LangGraph.

Best Practices & Anti-Patterns

Best Practices

- ✓ Start simple, add incrementally
- ✓ One responsibility per step
- ✓ Test sub-chains independently
- ✓ Use `assign()` for context accumulation
- ✓ Add fallbacks at model level
- ✓ Name chains for observability
- ✓ Separate prompts from logic
- ✓ Document I/O contracts
- ✓ Use streaming for user-facing chains
- ✓ Monitor token usage in LangSmith

Anti-Patterns to Avoid

- ✗ **God Chain:** Break into focused sub-chains
- ✗ **Prompt Stuffing:** Multi-step chains with focused prompts
- ✗ **Missing Error Handling:** Add `.with_retry()` and `.with_fallbacks()`
- ✗ **Hardcoded Models:** Use `configurable_alternatives()`
- ✗ **Ignoring Token Limits:** Add token counting & truncation
- ✗ **Sequential for Parallel:** Use `RunnableParallel`
- ✗ **No Observability:** Enable LangSmith, name every chain
- ✗ **Reinventing Memory:** Use built-in memory modules

Start simple and iterate — begin with prompt → model → parser, test it, then add complexity one step at a time.

Key Takeaways

Key Takeaways

1

Chains Connect Components — LCEL pipe operator replaces rigid legacy classes. Every chain supports invoke, stream, batch, and tracing.

2

Five Category Mental Model — Sequential, Parallel, Conditional, Map-Reduce, and Recursive cover all workflow patterns.

3

Document Processing — Stuff (small), Map-Reduce (large/parallel), Refine (quality), Map-Rerank (single best answer).

4

Conversational + RAG — Memory integration + question reformulation = conversational RAG. Summary Buffer is the best default.

5

Compose & Observe — Build complex systems from simple tested blocks. Name chains, enable tracing, monitor token usage.

Chains are the bridge between individual components and production applications — master these patterns.

Memory Systems



For Conversational Applications

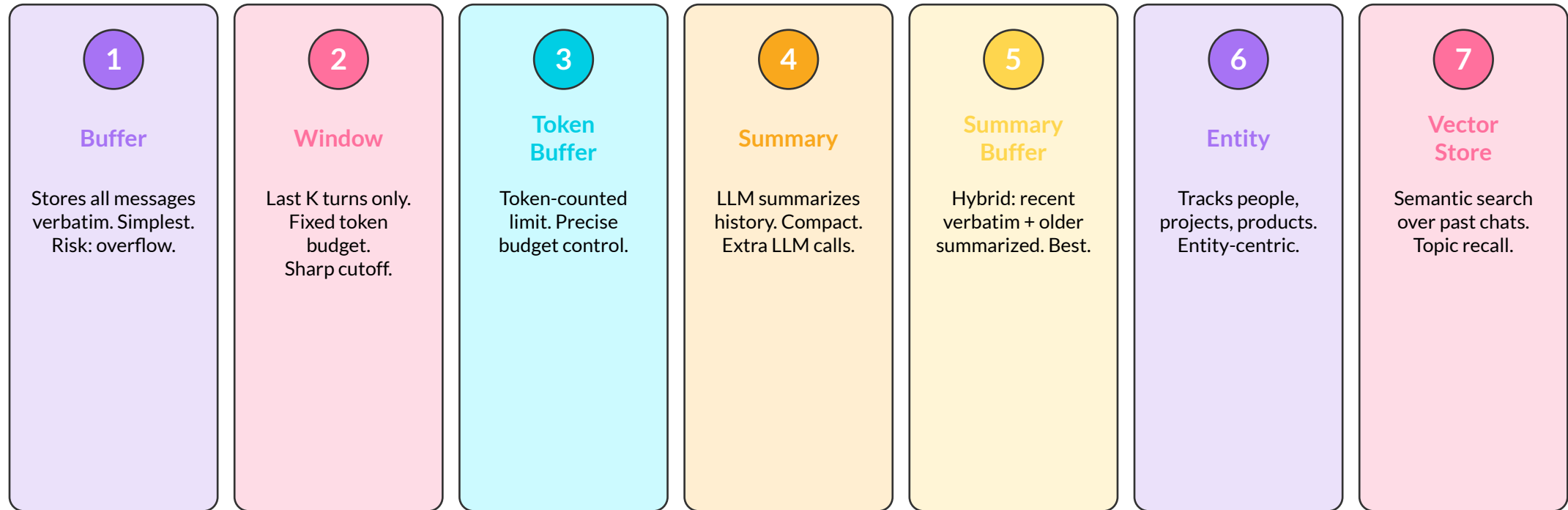
Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

Seven Memory Types – Overview

LLMs are stateless – memory systems store, manage, and re-inject conversation history into each prompt.



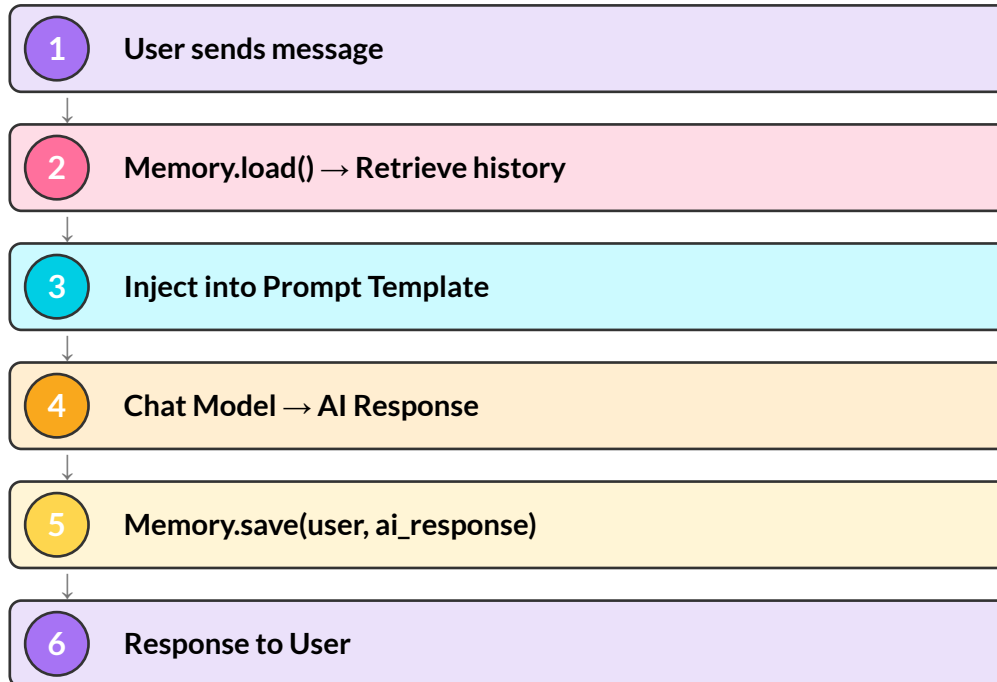
Memory transforms stateless LLMs into seemingly stateful conversational partners.

How Memory Works



Memory Lifecycle & Architecture

Two-Phase Pattern: Load history before the LLM call, save the exchange after. Every memory type follows this.



Prompt Template Structure

```
# Fixed instructions
SystemMessage(instructions)

# Dynamic memory slot
MessagesPlaceholder("chat_history")
← Memory injects here

# Current user query
HumanMessage(user_input)
```

MessagesPlaceholder is the dynamic slot where memory injects previous messages. Every conversational chain uses this pattern.

Buffer & Window Memory

Buffer vs Window Memory

1 ConversationBufferMemory

Storage: Complete message list

Growth: Linear — grows every turn

Info Loss: Zero — perfect recall

Overhead: None — just list append

Risk: Context window overflow

The Overflow Problem

```
# 8K context window, 200 tok/turn
After 30 turns: ~6,000 tokens
After 40 turns: ~8,000 tokens ⚠ Full
After 41 turns: FAILS ✖
```

2 ConversationBufferWindowMemory

Storage: Returns last K pairs only

Growth: Fixed — always $K \times 2$ messages

Info Loss: Moderate — older context lost

Overhead: Minimal — list slicing

Risk: Low — predictable budget

Choosing K

K=3 ~600 → Quick Q&A bots

K=5 ~1,000 → Standard chatbots

K=10 ~2,000 → Detailed conversations

K=15 ~3,000 → Complex support

K=20 ~4,000 → Maximum practical

Buffer = perfect recall but will overflow. Window = bounded but sharp cutoff. Both are free (no extra LLM calls).

Summary & Summary Buffer

Summary vs Summary Buffer

4 ConversationSummaryMemory

```
# Turn 1:  
"Looking for a laptop for editing"  
Summary: "User wants editing laptop"  
  
# Turn 2:  
"Budget is $2000"  
Summary: "User wants editing laptop,  
$2000 budget. Discussed MacBook Pro."  
  
# Summary grows slowly, never overflows
```

Growth: Slow — logarithmic

Info Loss: Some — nuance lost

Overhead: High — 1 LLM call/turn

Best For: Long conversations (50+)

5 SummaryBufferMemory ★

```
# Summary of older messages:  
"Video editor, $2000 budget. Explored  
MacBook Pro 14 and Dell XPS 15."  
  
# Recent messages (verbatim):  
Turn 8: "What about warranty?"  
Turn 9: "Can I upgrade RAM?"  
Turn 10: "I'll go with Dell XPS"  
  
# Hybrid: summary + exact recent
```

Growth: Very slow — bounded & efficient

Info Loss: Minimal — recent exact + old summarized

Overhead: Medium — occasional summary calls

Best For: General purpose — best default

Tip: Use a cheaper model (GPT-3.5, Haiku) for summary updates while using a powerful model for the main conversation.

Summary Buffer = best general-purpose choice. Exact recent messages + summarized older context = best of both worlds.

Entity & VectorStore Memory

Entity & VectorStore Memory

6 Entity Memory

```
Entities:
"Project Alpha": "Mobile app,
  due March. Team of 5. React Native."

"Sarah": "PM for Project Alpha.
  Based in Chennai. Weekly standups."

"React Native": "Framework v0.72"
```

Growth: Per-entity, not per turn

Overhead: High – 2 LLM calls/turn

Best For: CRM, project management, preferences

Limitation: Entity resolution can be imperfect

7 VectorStoreMemory

```
Stored as vectors:
[0.23,0.45...] → "Python data types"
[0.67,0.12...] → "Java experience"
[0.56,0.34...] → "Database design"

Query: "How to structure tables?"
→ Retrieves: "Database design" (most similar)
```

Retrieval: Constant – top-K semantically similar

Overhead: Medium – embedding + search per turn

Best For: Very long chats, topic revisits, multi-session

Limitation: No chronological awareness

Entity = structured knowledge about people/projects. VectorStore = semantic search over all past conversations.

Memory Type Comparison

Memory Type Comparison

Memory Type	Token Growth	Info Loss	LLM Overhead	Best For	Worst For
Buffer	Unbounded	None	None	Short chats, prototyping	Long conversations
Window (K)	Fixed	Sharp cutoff	None	Medium chats	Needing early context
Token Buffer	Fixed	Sharp cutoff	None	Variable-length msgs	Long chats needing history
Summary	Slow growth	Moderate	1 LLM call/turn	Long conversations	Short chats (overhead)
Summary Buffer ★	Very slow	Minimal	Occasional	General purpose – best	Very short chats
Entity	Per-entity	Some	2 LLM calls/turn	Entity-centric chats	Narrative conversations
VectorStore	Constant retrieval	Retrieval-dep.	Embedding/turn	Very long, topic-revisit	Sequential, recent-context

Default recommendation: Start with ConversationSummaryBufferMemory – handles the widest range of scenarios well.



Choosing the Right Memory

Step 1: Conversation Length

- 1–15 turns → BufferMemory
- 15–50 turns → Window / TokenBuffer
- 50+ turns → SummaryBufferMemory

Step 2: Context Requirements

- Exact recent messages? → Buffer, Window, Summary Buffer
- Old context preserved? → Summary, Summary Buffer, Entity
- Topic-based recall? → VectorStore

Step 3: Cost Constraints

- Minimal budget → Buffer or Window (no extra LLM calls)
- Moderate budget → Summary Buffer (occasional summaries)
- Generous budget → Entity or VectorStore

Step 4: Conversation Pattern

- Linear discussions → Buffer or Window
- Entity-centric (people, products) → Entity Memory
- Topic-switching with revisits → VectorStore
- General / uncertain → Summary Buffer

Default: Start with SummaryBufferMemory. Switch to specialized types only if needed.

Persistent Memory & Advanced Patterns



Persistent Memory

Production Requirement: In-memory storage is for dev only. Memory must be persisted to external storage.

Backend	Use Case	Advantages	Limitations
Redis	Fast session-based	Sub-ms reads, TTL support	In-memory (needs config)
PostgreSQL	Structured storage	ACID, rich querying	Slower for simple lookups
MongoDB	Flexible documents	Schema-flexible	Separate infrastructure
DynamoDB	Serverless apps	Auto-scaling, pay-per-req	AWS lock-in

Cross-Session Memory Strategies

1

Session Summary

Generate summary at session end, load as context for new sessions.

2

Entity Persistence

Persist entity knowledge base across sessions. Preferences survive.

3

Vector Persistence

Store all exchanges as vectors. New sessions retrieve from any session.

Key memory by session ID. Never share memory across unrelated conversations.



Advanced Memory Patterns

1

Multi-Memory

Combine multiple memory types simultaneously:

- Window (K=5) for recent
- Entity for structured data
- VectorStore for semantic

All inject into one prompt.
Most powerful but complex.

2

Token Budget

Dynamically adjust memory based on available space:

Total context:	8,000
System prompt:	-500
RAG documents:	-2,000
Current message:	-200
Response reserve:	-2,000
Memory budget:	3,300

3

Filtered Memory

Apply rules before storing:

- ✓ Privacy: Remove PII
- ✓ Relevance: Skip small talk
- ✓ Confidentiality: Exclude sensitive data (passwords)
- ✓ Deduplication: Avoid repetitive storage

4

Memory-Aware Prompts

System prompts that reference memory structure:

"You have access to:
{entity_memory}"

Recent exchanges:
{recent_history}"

Related past discussions:
{vector_memory}"

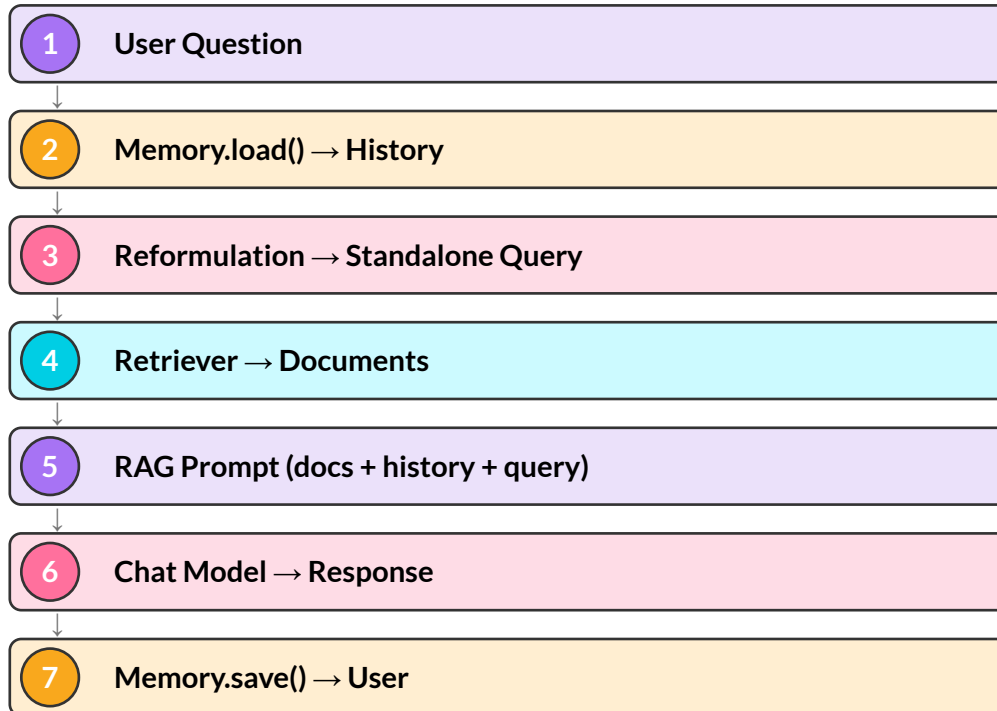
Advanced patterns solve production challenges: budget management, privacy compliance, and multi-memory fusion.

Memory for Conversational RAG



Conversational RAG Architecture

Most Critical Pattern: Combining memory with RAG. Question reformulation is mandatory for follow-up questions.



Token Budget Allocation

System prompt



Conversation history



Retrieved documents



Current question



Response reserve



Why Reformulation?

"Who created it?" → retriever fails

"Who created LangChain?" → finds docs ✓

Best Practices & Pitfalls

Best Practices & Common Pitfalls

Best Practices

- ✓ Start with Summary Buffer as default
- ✓ Set explicit token budgets (max_token_limit)
- ✓ Use cheaper models for memory ops
- ✓ Persist memory in production (Redis/Postgres)
- ✓ Reformulate queries in conversational RAG
- ✓ Key memory by unique session ID
- ✓ Test with 50+ turn conversations
- ✓ Monitor memory token usage in LangSmith
- ✓ Filter sensitive data (PII) before storing
- ✓ Plan cross-session memory early

Common Pitfalls

- ✗ **Buffer for long chats:** Use Summary Buffer or Window
- ✗ **No memory in conv. RAG:** Add question reformulation
- ✗ **Ignoring token budgets:** Set explicit max_token_limit
- ✗ **Same model for summaries:** Use cheaper model for memory ops
- ✗ **No session isolation:** Key memory by unique session ID
- ✗ **No persistence:** Persist to Redis/PostgreSQL
- ✗ **Storing everything (PII):** Apply filtering before save
- ✗ **No memory testing:** Include 50+ turn tests in QA

Memory bugs only surface after many turns — always test with 50+ turn conversations.

Key Takeaways

Key Takeaways

1

LLMs Are Stateless — Memory systems store, manage, and re-inject conversation history. Without memory, every API call is a first conversation.

2

Seven Types, One Default — Buffer, Window, Token Buffer, Summary, Summary Buffer, Entity, VectorStore. Start with Summary Buffer as the best general-purpose choice.

3

Persist for Production — In-memory is dev only. Use Redis for fast session-based memory or PostgreSQL for structured long-term storage.

4

Reformulate for RAG — Follow-up questions with pronouns will fail at retrieval. Question reformulation into standalone queries is mandatory.

5

Budget & Test — Token budget management is critical — memory, RAG, system prompts all compete for context. Test with 50+ turn conversations.

Memory transforms stateless LLMs into seemingly stateful conversational partners — master these patterns for production.

Document Loaders & Text Splitters



From Raw Data to LLM-Ready Chunks

Module 1: Foundations & LLM Essentials • Section 3.3

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover

Building the data ingestion pipeline — from raw files to optimally-sized, LLM-ready chunks

1

Document Loaders

Extracting text & metadata from any source into Document objects

2

File & Web Loaders

PDFs, Word, CSV, web pages, platforms, and cloud storage

3

Text Splitter Fundamentals

Why splitting matters, the dilemma, and splitter architecture

4

7 Splitter Types

Recursive, Character, Token, Language, Markdown, HTML, Semantic

5

Optimization & Practices

Decision framework, chunk sizing, two-pass strategy, pitfalls

Document Loaders



Document Loaders – The Foundation

Core Problem: Raw data lives in PDFs, Word docs, websites, and databases. LLMs understand text tokens. Document loaders bridge this gap.

The Document Object

```
Document(  
    page_content = "The extracted text..."  
    metadata = {source, page, url, ...}  
)
```

Field	Type	Description
page_content	string	The extracted text content
metadata	dict	Source info (filename, page, URL, timestamp)

Every loader produces this same standardized output regardless of data source.

Why Metadata Matters

Source Attribution

Which document answered the question

Filtering

Search by department, date, or author

Access Control

Enforce permissions by classification



Freshness Tracking

Know when source was last updated

Debugging

Trace failures back to source document

Always preserve and enrich metadata. The best RAG systems treat metadata as a first-class concern.

File-Based & Web Loaders

1 PDF Loaders

Challenge: PDFs are the most common enterprise format and the most challenging to parse. No single loader works for all PDFs.

Loader	Library	Strengths	Weaknesses
PyPDFLoader	PyPDF2	Fast, lightweight, page-by-page	Struggles with complex layouts, tables
PyMuPDFLoader	PyMuPDF	Fast, good extraction, rich metadata	Requires system dependencies
UnstructuredPDF	Unstructured	Handles complex layouts, tables, images	Slower, heavier dependency
PDFPlumberLoader	pdfplumber	Excellent table extraction, precise	Slower for large documents
AmazonTextract	AWS Textract	OCR for scanned PDFs, enterprise-grade	Requires AWS, costs per page

How to Choose

Simple text PDFs → PyPDFLoader (fastest, lightest)

Complex layouts → UnstructuredPDFLoader

Table-heavy docs → PDFPlumberLoader

Scanned PDFs → AmazonTextractPDFLoader or OCR

2

Other File & Web Loaders

File Loaders

Loader	Format	Key Features
Docx2txtLoader	.docx	Lightweight, text-only extraction
CSVLoader	.csv	Each row → Document, columns as metadata
TextLoader	.txt	Plain text files. Simplest loader
MarkdownLoader	.md	Markdown with header-based splitting
JSONLoader	.json	jq-style path extraction for nested data
ExcelLoader	.xlsx	Spreadsheet with sheet awareness
ImageLoader	.png/.jpg	OCR-based text extraction from images

Web Page Loaders

Loader	Best For
WebBaseLoader	Simple static HTML (BeautifulSoup)
UnstructuredURL	Complex content-heavy pages
SeleniumURL	JS-rendered SPAs, dynamic content
PlaywrightURL	Modern JS-heavy sites
RecursiveUrl	Crawl a site with depth control

Platform-Specific Loaders

Notion, Confluence, Google Drive — Productivity & collaboration tools

Git, GitHub Issues, Wikipedia, Arxiv — Developer & knowledge platforms

S3, GCS, Azure Blob Storage — Cloud storage integrations

Slack, Discord — Messaging platform history

LangChain provides 100+ loaders covering files, web pages, platforms, databases, and cloud storage.

Text Splitters



Why Text Splitting Matters

The Problem: Documents can be hundreds of pages, but embedding models have size limits, context windows are finite, and retrieval precision improves with smaller chunks.

Why Split Documents?

Embedding Limits: Models accept 512–8192 tokens max

Context Windows: Can't pass an entire book as context

Retrieval Precision: 500-word focused chunk > 5000-word chapter

Cost Optimization: Smaller chunks = fewer tokens per query

The Splitting Dilemma

Chunks Too Small

- ✗ Lose context — fragments meaningless alone
- ✗ Embedding quality degrades
- ✗ Retrieval returns useless snippets

Chunks Too Large

- ✗ Embedding becomes too general
- ✗ Retrieval precision drops
- ✗ Wastes context window space

Why Overlap Matters

Without Overlap: Sentence split across chunks — info lost at boundary!

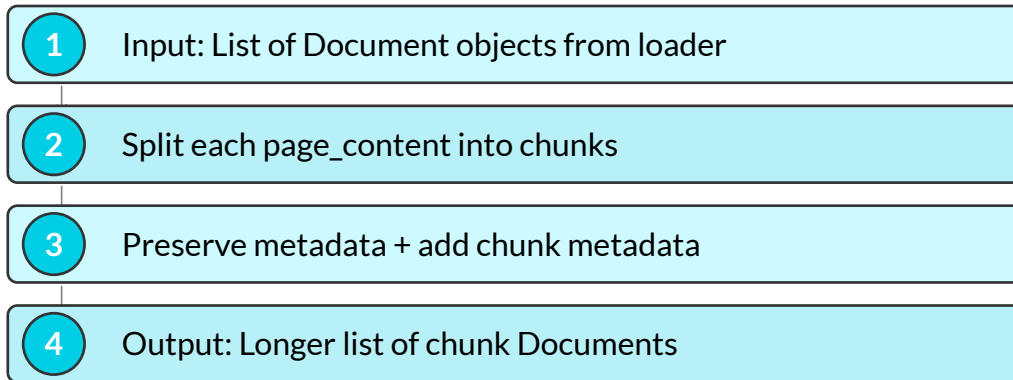
With Overlap (10–20%): Boundary content in both chunks — no info lost!

The quality ceiling of your entire LLM application is set at the document loading and splitting stage.



Splitter Architecture & Key Parameters

How All Splitters Work



Key Parameters

Parameter	Description	Typical
chunk_size	Max chunk size	500-2000
chunk_overlap	Overlap between chunks	50-200
length_function	How to measure size	len or tokens
separators	Ordered boundary list	Varies

RecursiveCharacterTextSplitter – The Recommended Default

Tries the largest meaningful boundary first, falling back to smaller separators. Merges small chunks, applies overlap.



Start with RecursiveCharacterTextSplitter (chunk_size=1000, chunk_overlap=200) for most use cases.



Specialized Splitter Types



TokenText Splitter

What: Splits by token count using a tokenizer (tiktoken for OpenAI models).

Why: Characters $\neq$ tokens. "Hello" = 1 token. "こんにちは" = 3 tokens. Token-based gives precise control.

When: Exact token budget, multilingual content, code-heavy text, paired with a model's tokenizer.



Language Code Splitters

What: `from_language()` — splits at programming language syntax boundaries.

How: Uses class, def, function, struct boundaries instead of generic newlines.

Supports: Python, JS, TS, Java, Go, Rust, Ruby, Markdown, HTML, LaTeX.

Why: Generic splitters break mid-function, separating signature from body.



Semantic Chunker

What: Uses embedding similarity to find natural topic boundaries.

How: Embed each sentence $\rightarrow$ cosine similarity $\rightarrow$ split where similarity drops significantly.

Trade-off: Highest quality but slow (embeds every sentence). No size guarantee — combine with size-based second pass.

Splitter Comparison – Complete Reference

Splitter	Split Logic	Semantic	Speed	Size Ctrl	Best For
RecursiveChar	Hierarchical separators	Low	Very fast	Precise	General – best default
Character	Single separator	None	Very fast	Limited	Simple, uniform text
Token	Token count	None	Fast	Exact	Token-critical apps
Language	Syntax boundaries	Medium	Fast	Good	Source code
MarkdownHeader	Header hierarchy	High	Fast	Variable	Markdown docs
HTMLHeader	HTML header tags	High	Fast	Variable	Web content
Semantic	Embedding similarity	Highest	Slow	None	Transcripts, mixed text

Default: RecursiveCharacterTextSplitter. Specialize only when your content type demands it.

Choosing & Optimizing



Decision Framework & Chunk Sizes

Step 1: Content Type?

Source code → Language-specific code splitter

Markdown → MarkdownHeaderTextSplitter (+Recursive)

HTML → HTMLHeaderTextSplitter (+Recursive)

General text → Continue to Step 2 →

Step 2: Semantic Precision?

Critical → SemanticChunker (+size second pass)

Standard → RecursiveCharacterTextSplitter

Not important → CharacterTextSplitter

Step 3: Exact Token Control?

Yes → TokenTextSplitter or token length_function

No → Character-based splitting is fine

Chunk Size by Use Case

Use Case	Size	Overlap
General Q&A	500–1000	100–200
Detailed analysis	1000–2000	200–400
Code search	1000–1500	100–200
Legal / medical	500–800	150–200
Conversational	200–500	50–100
Summarization	2000–4000	200–400

Two-Pass Splitting Strategy

P1 Structure-aware split (headers, sections)



P2 Size enforcement (RecursiveChar for oversized)



✓ Semantic coherence + size compliance

Chunk size optimization is empirical — start with defaults, test with actual queries, measure with RAGAS.

Best Practices & Pitfalls



Best Practices

✓ **Always inspect loaded documents:** Print the first few docs to verify content quality and metadata

✓ **Handle loading errors gracefully:** Use try/except and log failures for corrupt files

✓ **Deduplicate before splitting:** Remove duplicate documents to avoid redundant chunks

✓ **Preserve metadata through splitting:** Ensure splitter passes loader metadata to all child chunks

✓ **Add chunk-specific metadata:** Include chunk_index and total_chunks for reconstruction

✓ **Start with RecursiveCharacter:** It's the best default. Optimize to specialized splitters only if needed

✓ **Use two-pass for structured docs:** Header-based first, then size-based for oversized sections

✓ **Test chunk quality manually:** Read 20–30 random chunks. Do they make sense standalone?

✓ **Benchmark empirically:** Test 3–4 sizes with actual queries using RAGAS

✓ **Monitor document freshness:** Track when sources were last loaded and re-index on changes

Common Pitfalls

Pitfall	Problem	Solution
Wrong PDF loader	Tables as gibberish, broken layout	PDFPlumber for tables, Unstructured for complex
No overlap	Info lost at boundaries	Set overlap to 10–20% of chunk_size
Chunks too small	Snippets lack context	Increase chunk_size (min 300–500 chars)
Chunks too large	Low retrieval precision	Decrease chunk_size (max 1500–2000 chars)
Ignoring metadata	Can't trace answers to sources	Preserve and enrich metadata at every stage
Generic splitter for code	Functions split mid-definition	Use Language-specific code splitters
No content inspection	Silent extraction failures	Always sample and verify loaded documents
Over-engineering	Complex custom logic when defaults work	Start with RecursiveCharacter, optimize later

A perfectly tuned RAG pipeline will produce terrible results if document loading or splitting is done poorly.

Key Takeaways

Key Takeaways

- 1 Document loaders bridge raw data to LLMs:** 100+ loaders extract text and metadata from files, web, platforms, and cloud into standardized Document objects.
- 2 Metadata is a first-class concern:** `page_content` + metadata enable source attribution, filtering, access control, and debugging in production.
- 3 Text splitting sets your quality ceiling:** Chunks too small lose context; too large lose precision. Sweet spot: 500–1000 chars with 10–20% overlap.
- 4 RecursiveCharacterTextSplitter is your default:** Hierarchical separator approach works for the vast majority of use cases. Start with `chunk_size=1000`, `overlap=200`.
- 5 Two-pass splitting for structured docs:** Structure-aware first (headers), size enforcement second. Maximum semantic coherence within size constraints.

The quality ceiling of your entire LLM application is set at the document loading and text splitting stage.

Thank You!



Nunnari Academy

Generative AI & Agentic AI Program

Module 1: Foundations & LLM Essentials • Section 3.3